

## Упражнение 23

Одна из практических целей для задачи нахождения минимального остовного дерева — построение остовой сети для множества узлов с минимальной общей стоимостью. Однако сейчас нас интересует другой тип целей: проектирование остовой сети, в которой максимальная стоимость ребра настолько низка, насколько это возможно.

Говоря конкретнее, пусть  $G = (V, E)$  — связный граф с  $n$  вершинами,  $m$  ребрами и положительными стоимостями ребер (предполагается, что все стоимости различны). Пусть  $T = (V, E')$  — остовное дерево  $G$ ; *критическим ребром* дерева  $T$  будет называться ребро из  $T$  с наибольшей стоимостью.

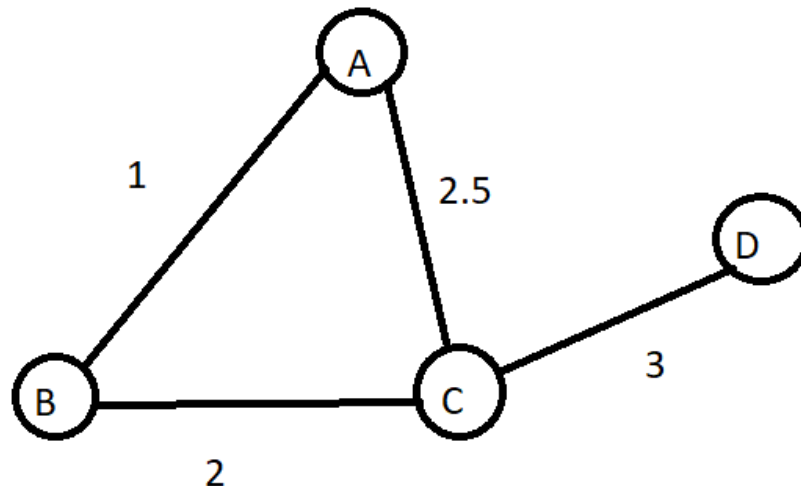
Остовное дерево  $T$  графа  $G$  называется *минимально-критическим остовным деревом* (minimum bottleneck spanning tree), если не существует остовного дерева  $T'$  графа  $G$  с меньшей стоимостью критического ребра.

(a) Является ли каждое минимально-критическое дерево графа  $G$  минимальным остовным деревом  $G$ ? Докажите или приведите контрпример.

(b) Является ли каждое минимальное остовное дерево графа  $G$  минимально-критическим деревом графа  $G$ ? Докажите или приведите контрпример.

**Пункт а:**

**Контрпример:**



1. Любое остовное дерево обязательно содержит ребро CD, иначе вершина D останется изолированной. Значит, в любом остовном дереве есть ребро веса 3, и меньшего максимального веса меньше 3 быть не может.

2. Все остовные деревья получаются так: берём CD и любые два ребра из треугольника ABC, которые соединяют A, B, C без цикла. Возможны три варианта:

$\{AB, BC, CD\}$  - веса 1, 2, 3

$\{AB, AC, CD\}$  - веса 1, 2.5, 3

$\{BC, AC, CD\}$  - веса 2, 2.5, 3

В каждом дереве максимальное ребро равно 3. Следовательно, каждое из этих трёх деревьев является минимально-критическим потому что меньше 3 сделать нельзя.

3. Минимальное остовное дерево по сумме - это то, у которого сумма весов наименьшая. Считаем суммы:

Первое дерево:  $1 + 2 + 3 = 6$

Второе:  $1 + 2.5 + 3 = 6.5$

Третье:  $2 + 2.5 + 3 = 7.5$

Наименьшую сумму имеет первое дерево. Оно и является единственным минимальным остовным деревом.

**4. Вывод: Два других дерева - минимально-критические, но не минимальные остовные. Значит, утверждение, что каждое минимально-критическое дерево является минимальным остовным - неверно.**

(b) Является ли каждое минимальное остовное дерево графа  $G$  минимально-критическим деревом графа  $G$ ? Докажите или приведите контрпример.

**Ответ: Да, является**

**Доказательство:**

Пусть  $T = (V, E')$  - минимальное остовное дерево графа  $G$ . Пусть  $e$  - критическое ребро дерева  $T$ , то есть ребро из  $E'$  с наибольшей стоимостью. Обозначим его стоимость через  $c$ .

Предположим, что существует другое остовное дерево  $T' = (V, E'')$  графа  $G$ , у которого критическое ребро имеет стоимость меньше  $c$ . Тогда все рёбра из  $E''$  дешевле  $c$ .

Удалим из  $T$  ребро  $e$ . Дерево  $T$  распадётся на две компоненты связности. Дерево  $T'$  связно, поэтому в нём найдётся ребро  $f$ , соединяющее эти две компоненты. Стоимость ребра  $f$  меньше  $c$ .

Теперь возьмём дерево  $T$ , уберём из него ребро  $e$  и добавим ребро  $f$ . Получится новое остовное дерево графа  $G$ . Его общая стоимость меньше, чем у  $T$ , потому что мы заменили ребро стоимостью  $s$  на более дешёвое ребро. Это противоречит тому, что  $T$  - минимальное остовное дерево.

**Значит, наше предположение неверно. Следовательно, не существует остовного дерева, у которого критическое ребро дешевле, чем у  $T$ . То есть  $T$  является минимально-критическим остовным деревом.**

## Упражнение 24

Пусть  $G = (V, E)$  — (ненаправленный) граф со стоимостями  $c_e \geq 0$  ребер  $e \in E$ .

Допустим, вам известно минимальное остовное дерево  $T$  для  $G$ . Предположим, что в  $G$  добавляется новое ребро, соединяющее два узла  $v, w \in V$  со стоимостью  $s$ .

(а) Предложите эффективный алгоритм проверки того, остается ли  $T$  остовным деревом минимальной стоимости при добавлении нового ребра в  $G$  (но не в дерево  $T$ ). Алгоритм должен выполняться за время  $O(|E|)$ . Удастся ли вам заставить его выполняться за время  $O(|V|)$ ? Отметьте все предположения, которые вам пришлось сделать относительно структуры данных, используемой для представления дерева  $T$  и графа  $G$ .

**Пункт А:**

**Алгоритм.**

1. Найти в дереве  $T$  путь между вершинами  $v$  и  $w$ .
2. На этом пути найти ребро  $e_{\max}$  с максимальным весом. Обозначить его вес через  $w_{\max}$ .
3. Сравнить вес нового ребра  $s$  с  $w_{\max}$ :

Если  $s \geq w_{\max}$ , то  $T$  остаётся минимальным остовным деревом.

Если  $s < w_{\max}$ , то  $T$  перестаёт быть минимальным.

**Время работы**

Вариант 1.  $O(|E|)$ .

Можно просто обойти весь граф  $G$  или все рёбра  $T$ , но это неоптимально.

## Вариант 2. $O(|V|)$

В дереве  $T$  ровно  $|V| - 1$  ребро. Поиск пути между двумя вершинами в дереве можно выполнить с помощью обхода в глубину или в ширину за  $O(|V|)$ , так как дерево - связный граф без циклов. Нахождение максимального ребра на пути также занимает  $O(|V|)$ .

Таким образом, алгоритм можно реализовать за  $O(|V|)$ .

### Предположения о структуре данных

Чтобы алгоритм работал за  $O(|V|)$ , необходимо:

1. Дерево  $T$  должно быть представлено в виде списка смежности. Это позволяет за  $O(|V|)$  выполнить обход от  $v$  до  $w$ .
2. Для каждого ребра должен быть доступен его вес.

**Ответ: Да, можно сделать за  $O(|V|)$ , при условии, что дерево  $T$  представлено в подходящей структуре данных.**

Пусть  $G = (V, E)$  — (ненаправленный) граф со стоимостями  $c_e \geq 0$  ребер  $e \in E$ .

Допустим, вам известно минимальное остовное дерево  $T$  для  $G$ . Предположим, что в  $G$  добавляется новое ребро, соединяющее два узла  $v, w \in V$  со стоимостью  $c$ .

(b) Допустим,  $T$  уже не является минимальным остовным деревом. Предложите алгоритм с линейным временем ( $O(|E|)$ ) для замены  $T$  новым минимальным остовным деревом.

### Пункт Б:

Сначала выполняем алгоритм из пункта (a):

1. находим в  $T$  путь между вершинами  $v$  и  $w$
2. Находим на этом пути ребро  $e_{\max}$  с наибольшим весом  $w_{\max}$
3. Сравниваем  $c$  и  $w_{\max}$ .
4. Так как  $T$  не минимально, то  $c < w_{\max}$ .

Теперь строим новое минимальное остовное дерево:

1. Удаляем из  $T$  ребро  $e_{\max}$ .
2. Добавляем в  $T$  новое ребро  $(v, w)$ .

Полученное дерево является минимальным остовным деревом для графа с добавленным ребром.

**То есть пункт Б - это пункт А + замена ребра**

Время работы совпадает с временем работы алгоритма из пункта А и составляет  $O(|V|)$ .

## Упражнение 25

Предположим, имеется связный граф  $G = (V, E)$ , ребро  $e$  которого имело стоимость  $c_e$ . При различных стоимостях ребер  $G$  содержит уникальное минимальное остовное дерево. Но если в графе имеются ребра с одинаковой стоимостью,  $G$  может содержать несколько минимальных остовных деревьев.

Вопрос: можно ли заставить алгоритм Крускала найти все минимальные остовные деревья графа  $G$ ?

Вспомните, что алгоритм Крускала сортирует ребра в порядке увеличения стоимости, а затем жадно обрабатывает ребра одно за другим, добавляя ребро  $e$  при условии, что оно не приводит к образованию цикла. Когда некоторые ребра имеют одинаковую стоимость, выражение «в порядке увеличения стоимости» необходимо задать более тщательно: упорядочение ребер называется *действительным*, если соответствующая последовательность стоимостей ребер не уменьшается. *Действительным выполнением* алгоритма Крускала будет называться выполнение, начинающее с действительного упорядочения ребер  $G$ .

Для любого графа  $G$  и любого минимального остовного дерева  $T$  графа  $G$  существует ли действительное выполнение алгоритма Крускала для  $G$ , которое выдает  $T$  как результат? Приведите доказательство или контрпример.

53

**Ответ: Да, существует**

**Доказательство:**

Возьмём любое минимальное остовное дерево  $T$  графа  $G$ . Построим порядок рёбер для алгоритма Крускала следующим образом:

1. Сначала перечислим все рёбра, принадлежащие  $T$ , в порядке возрастания их весов.
2. Затем перечислим все остальные рёбра не принадлежащие  $T$  тоже в порядке возрастания их весов.

Если среди рёбер с одинаковым весом есть и рёбра из  $T$ , и рёбра не из  $T$ , то в нашем порядке сначала идут рёбра из  $T$ , а потом — все остальные рёбра того же веса.

**Запустим алгоритм Крускала с таким порядком рёбер.**

Все рёбра из  $T$  будут добавлены, потому что  $T$  не содержит циклов.

-Рассмотрим любое ребро  $f$ , не входящее в  $T$ . Пусть его вес равен  $w$ . В дереве  $T$  существует единственный путь между концами  $f$ . Все рёбра этого пути имеют вес не больше  $w$ , поскольку иначе  $T$  не было бы минимальным остовным деревом. К моменту обработки  $f$  все рёбра этого пути уже добавлены, так как они либо легче  $w$ , либо имеют тот же вес  $w$ , но идут раньше  $f$ , потому что рёбра из  $T$  идут раньше остальных рёбер того же веса. Значит, концы  $f$  уже находятся в одной компоненте, и добавление  $f$  создало бы цикл. Поэтому алгоритм не добавит  $f$ .

Таким образом, алгоритм Крускала добавит все рёбра из  $T$  и не добавит ни одного ребра не из  $T$ . В результате получится дерево  $T$ .